

Eseguire WiFiCatcher senza processo interamente root

Proposta di design · separazione dei privilegi · C (helper privilegiato) con A+ come alternativa più leggera

Stato: proposta / su carta. Nessun codice ancora modificato. **Scopo:** spiegare bene il modello **C** (helper privilegiato) — prima ad alto livello, poi in dettaglio con le tre modalità di avvio — e confrontarlo con l'alternativa più leggera **A+** (wrapper `sudo` validato). L'obiettivo è ridurre al minimo l'onere di setup e le modifiche al sistema operativo. Nessuna backward compatibility con l'attuale "`sudo` su tutta l'app": l'app gira **sempre** come utente normale.

1. Obiettivi e vincoli

1. **Mai l'intera app come root.** Server web, parsing dei CSV, upload dei file e codice del grafo devono girare da utente normale. Un bug lì non deve essere un bug root.
2. **Gestire bene le poche funzioni realmente privilegiate**, senza elevare tutto.
3. **Minimo sforzo per l'utente / minime modifiche al SO.** Idealmente non deve toccare `sudoers`, installare servizi systemd o fare `setcap` solo per usare il tool.

2. Cosa richiede davvero root

WiFiCatcher non apre mai raw socket in Python: ogni azione privilegiata è un binario esterno lanciato via `subprocess`. Inventario verificato:

Funzione	Comando/i	Serve root?
Monitor mode on/off	<code>airmon-ng check kill</code> / <code>start</code> / <code>stop</code>	sì (netlink / interfaccia)
Cattura live	<code>airodump-ng ...</code>	sì (radio)
Deauth	<code>aireplay-ng --deauth ...</code>	sì (iniezione frame)
Enumerazione EAP	<code>EAP_buster.sh</code> → <code>wpa_supPLICant</code>	sì (si associa, usa l'interfaccia)
Ripristino rete all'uscita	<code>systemctl restart NetworkManager</code> / <code>service ...</code>	sì
Estrazione certificato	<code>tshark -r <cap></code> / <code>pcapFilter.sh</code>	no — legge un file
Detection handshake / WPS	<code>tshark -r <cap></code>	no — legge un file
Elenco interfacce	<code>/sys/class/net</code> (sysfs)	no
Web UI, parsing, upload, grafo	Python puro	no

Quindi servono root **solo cinque** famiglie di comandi. Tutto ciò che forma la vera superficie d'attacco (un server HTTP che accetta upload e lancia processi) **non** ne ha bisogno. È questo che rende la riduzione dei privilegi sia utile sia fattibile.

3. Modello di minaccia

Ci difendiamo da **un bug nella parte non privilegiata** (server web, gestore upload, parser) sfruttato per eseguire codice. Oggi quel codice girerebbe come root. L'obiettivo è che una compromissione simile dia, nel caso peggiore, i privilegi dell'utente dell'app più le poche azioni radio validate che esponiamo di proposito.

Non ci difendiamo da un root locale malevolo né dall'accesso fisico. Accettiamo anche che qualsiasi binario eseguito come root possa in linea di principio essere abusato se la sua superficie di argomenti è lasciata aperta — da qui l'enfasi sul **non** passare argomenti controllati dall'attaccante direttamente al comando.

4. Design C — ad alto livello

Due programmi che girano insieme, con un confine di fiducia netto:

- **L'app** (server web, UI, parsing, upload, grafo) gira come **utente normale**, mai root.
- **L'helper** (`wc-privhelperd`) è un programma **piccolo** che gira come **root** e fa **solo** le operazioni radio.

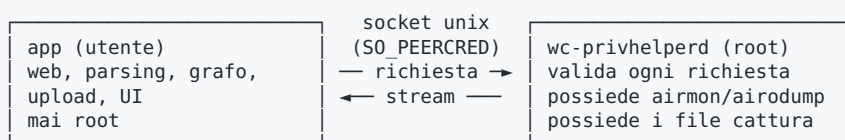
L'app **non esegue mai** i comandi radio. Quando serve un'azione privilegiata (accendere monitor mode, avviare una cattura, un deauth...), l'app **la chiede** all'helper attraverso un canale privato locale — un socket unix, in pratica una linea telefonica interna al PC, senza rete. L'helper riceve la richiesta, **la valida** e la esegue, poi restituisce il risultato.

Il punto chiave: l'app può solo **ordinare da un menù fisso** di operazioni (es. "deauth su questa rete, 5 pacchetti"), **non** inviare comandi arbitrari.

Analogia — il ristorante. L'app è il cameriere: non può entrare in cucina (root). L'helper è il cuoco, l'unico con i coltelli (i privilegi). Il cameriere passa un ordine scritto scelto da un menù fisso; il cuoco controlla che sia valido e cucina. Il cameriere non tocca mai i coltelli.

Conseguenza di sicurezza. Anche se qualcuno bucase il server web, **non otterrebbe root**: al massimo potrebbe chiedere all'helper quelle poche operazioni pre-approvate, con parametri controllati. Non può eseguire codice arbitrario come root, né lanciare altri binari.

5. Design C — in dettaglio



- **Argomenti validati, nessun pass-through.** L'helper **costruisce lui** ogni riga di comando a partire da campi validati. Un'app compromessa non può iniettare argomenti né eseguire altri binari. È la differenza sostanziale con A+ (vedi §7).
- **Streaming pulito delle catture.** L'helper lancia `airodump-ng`, possiede il suo CSV/pcap in rotazione, lo legge e **manda le righe sul socket**. L'app le interpreta come fa oggi, ma **non tocca mai file di proprietà root**. Sparisce del tutto il problema dei permessi dei file.
- **Autenticazione del socket.** Socket con permessi `0660`, proprietario `root:wificatcher`; in più l'helper verifica l'uid del chiamante via `SO_PEERCREDS` e rifiuta chiunque non sia l'utente atteso. Niente porte TCP, nessuna esposizione di rete.
- **Cleanup robusto.** Siccome l'helper possiede lo stato della radio, può rimettere l'interfaccia in managed mode e riavviare NetworkManager **quando l'app si disconnette**, anche se l'app va in crash — più robusto di oggi, dove il cleanup è legato al processo dell'app.

Contro: più codice (protocollo IPC, serializzazione, gestione errori, ciclo di vita, backpressure dello stream); due cose da avviare.

6. Le tre modalità di avvio — è qui che si decide l'impatto sul SO

La logica dell'helper è sempre la stessa: cambia solo **come ottiene i privilegi root all'avvio**. Tre modi, in ordine di "permanenza" crescente:

Modalità	Cosa installa l'utente	Prompt	Impronta sul SO
On-demand (default)	niente	1 password <code>sudo</code> a sessione	zero modifiche permanenti
sudoers NOPASSWD	1 riga in <code>sudoers.d</code>	nessuno	un piccolo file
unit systemd	1 file <code>.service</code> (+ socket)	nessuno	servizio, robusto, produzione

1. On-demand (default, zero installazione). L'app parte da utente. La prima volta che serve un'operazione radio, **è l'app a lanciare l'helper** con un singolo `sudo`:

```
# concettualmente
subprocess.Popen(["sudo", helper, "--socket", sock, "--peer-uid", str(os.getuid())])
```

L'utente inserisce la password **una volta per sessione**; l'helper resta vivo finché vive l'app e muore con essa. **Niente sudoers, niente systemd, niente lasciato sul sistema**. È la modalità che rispetta il vincolo "minime modifiche al SO".

2. sudoers (una riga, senza password). Aggiungi **una riga** che autorizza a lanciare solo l'helper senza password:

```
# /etc/sudoers.d/wificatcher
%wificatcher ALL=(root) NOPASSWD: /opt/wificatcher/wc-privhelperd
```

Tutto come on-demand, ma **senza il prompt** ogni sessione. Comodo se lo usi spesso. Costo: un piccolo file permanente.

3. systemd (servizio sempre attivo, per produzione). L'helper gira come **servizio di sistema** in background, sempre pronto. L'app non lo lancia: si limita a **connettersi** al socket già attivo. Ideale per deployment/server (parte al boot, gestito da systemd, log centralizzati). Costo: installi un file `.service`.

In sintesi: **stesso helper, tre modi di dargli root**. Di default on-demand (niente da installare, una password a sessione); chi non vuole il prompt usa la riga sudoers; in produzione si usa systemd.

7. A+ — l'alternativa più leggera (prima tappa)

Se vogliamo un risultato prima, esiste una versione ridotta: l'app resta unprivileged e instrada le operazioni **una-tantum** (deauth, monitor on/off, restart rete, enumerazione EAP) attraverso **un solo wrapper** validato, chiamato via `sudo`:

```
app (utente) — sudo -n wc-privhelper <op> --json '{...}' → wrapper (root)
                                                    valida, esegue
```

Il `sudoers` autorizza **un solo comando**, non i binari grezzi. Il wrapper è, di fatto, **il gestore di richieste dell'helper C senza il socket, il demone e lo streaming**: stesso nucleo di validazione. Buono per le operazioni una-tantum; inadatto alla cattura live (processo continuo), che è proprio dove serve C.

A+ e C convergono: A+ è "la C senza demone persistente e senza streaming". Chiudi già l'iniezione di argomenti; C aggiunge lo streaming pulito, il confine di processo vero e il cleanup robusto.

8. Sketch del protocollo (design C)

Messaggi JSON con prefisso di lunghezza sul socket. Set di operazioni fisso; ogni campo validato prima di eseguire qualsiasi cosa.

```

→ {"op": "monitor.start", "iface": "wlan0"}
← {"ok": true, "monitor_iface": "wlan0mon"}

→ {"op": "capture.start", "iface": "wlan0mon", "channel": 6, "bssid": null, ...}
← {"ok": true, "capture_id": "c1"}
← {"stream": "c1", "csv": "<snapshot CSV di airodump>"} # ripetuto ~1/s
← {"stream": "c1", "csv": "..."}

→ {"op": "capture.stop", "capture_id": "c1"}
← {"ok": true, "saved_path": "/home/utente/captures/..."} # chown all'utente app

→ {"op": "deauth", "iface": "wlan0mon", "bssid": "AA:...", "client": null, "count": 5}
← {"ok": true, "returncode": 0}

→ {"op": "eap.enumerate", "iface": "wlan0", "essid": "corp", "identity": "DOM\\u"}
← {"ok": true, "methods": [...]}

→ {"op": "network.restart"}
← {"ok": true}

```

Le regole di validazione riusano codice esistente: `iface` deve comparire in `/sys/class/net` ed essere wireless; i MAC via `models.normalize_mac`; `count` limitato a `MAX_COUNT` (64); `channel` un intero in range; `essid` / `identity` con le regex già presenti. `op` sconosciuta, campo non valido o uid del peer errato → rifiuto netto, loggato.

Estrazione certificato, handshake e WPS restano **nell'app** (leggono solo file); per una cattura live l'helper restituisce una copia leggibile o un percorso, così l'app esegue `tshark` da utente.

9. Impatto sul codice

- L'app gira **sempre** come utente: non esiste più un percorso "root su tutta l'app". Un solo modello, più semplice da ragionare.
- Il gate `operations/base.offensive_available()` passa da "sono root?" a "**l'helper è raggiungibile/avviabile?**".
- I corpi privilegiati in `operations/` e `capture/` sono già isolati: diventano l'implementazione dell'helper quasi as-is. Il lato app ottiene stub client sottili che serializzano la richiesta.

10. Raccomandazione

Adottare **il design C con avvio on-demand come default**, perché è l'unico che soddisfa tutti e tre i vincoli insieme:

- non esegue mai l'app come root ✓
- gestisce le cinque funzioni privilegiate tramite una superficie piccola, validata e auditabile ✓
- **zero modifiche permanenti al SO** nella modalità default (una password a sessione), con opt-in `sudoers` /systemd per chi non vuole il prompt ✓

A+ resta una tappa intermedia valida: possiamo spedire prima il wrapper validato `wc-privhelper` (è il gestore di richieste dell'helper senza socket/demone/ streaming) per le operazioni una-tantum, e farlo poi crescere nel demone on-demand per la cattura live.

11. Migrazione a fasi

1. **Estrarre e validare.** Raccogliere i costruttori dei comandi privilegiati in un modulo unico con validazione stretta per operazione (condiviso da tutte le fasi). Nessun cambio di comportamento.
2. **Wrapper A+.** Spedire `wc-privhelper <op>`; instradare le quattro operazioni una-tantum via `sudo -n` (fallback: `sudo` interattivo). Documentare la riga `sudoers` opzionale.
3. **Demone C + streaming.** Trasformare il wrapper in demone su socket; spostare la cattura live su CSV in streaming; aggiungere auth `SO_PEERCREG` e cleanup alla disconnessione.
4. **Deploy.** Fornire un'unit systemd opzionale per la produzione; on-demand come default.

12. Domande aperte / rischi

- `airmon-ng` è uno script di shell, quindi `setcap` è escluso — un motivo in più per passare dall'helper invece che dalle capabilities.
- **Un bug di validazione nell'helper resta un bug root**, ma su una superficie minuscola e fissa — è il compromesso che scegliamo esplicitamente.
- **UX del prompt on-demand:** dove chiedere la password quando l'app parte da un contesto browser; probabilmente solo da terminale, come oggi.
- **Proprietà delle catture salvate:** l'helper deve fare `chown` all'utente app, così i dati restano suoi.
- **Percorsi per distribuzione** di `systemctl` / `service` / binari da risolvere, non hardcodati.